Decentralized Multi-Agent Reinforcement Learning in Electric Vehicle (EV) Charging

Nuo Lin, Lidia Berhe, Kathryn Forrest
Sprint 1 – 10/14/2026

Mission Statement

The Decentralized EV Charging uses a multi-agent reinforcement learning system as an optimization method to minimize charging costs while protecting user data sourced from individual EVs.

Allows EV owners to harness technology to save money, and results can be compared to a baseline to demonstrate savings.

User Stories

What does the user want or need?

1. EV Owner: As the EV Owner, I want to have my car fully charged at the cheapest possible rate so that I can save money and have it ready when I want to use it.

2. Charging Station Owner: As the charging station owner, I want to know how much use I get at different times and how competitive my costs are so I keep customers happy and am able to fulfill the amount of power they desire.

3. EV Developer/Manufacturer: As the EV manufacturer, I want to make sure my vehicles have the most competitive charging scheme that will benefit the customer by getting them the cheapest possible cost while meeting their charging requirements.

Progress in Sprint 1

A baseline EV charging simulation and visualization tool was set up to show one car charging.

A visualization tool and simulation was set up to show multiple cars charging at the same station, based only on the order of their arrivals.

Data was sourced from local utility companies to include real world estimates so the information can be useful to manufacturers.

Tools

1. **Eversource On/Off Peak Pricing History from July 2025 through June 2026**

   Allows us to run simulation on real data

2. **Draw.io vision board/project mapping**

   Project planning and mapping – allows for visual representation of product before any actual coding

3. **Python and Matplotlib**

   Team familiarity with Python paired with open-source python rendering tool to create simulation visualizations.

4. **pytest Testing Framework**

   Ease of writing unit tests

Progress

Base scenario: 1 EV at 0% SOC, 1 charger, cost to charge to 100% using Eversource pricing. Edit assumptions in ev_charging/config.py

config.py | run_single_ev.py

run_single_ev.py

```python
"""Base scenario: 1 EV at 0% SOC, 1 charger, cost to charge to 100%
using Eversource pricing. Edit assumptions in ev_charging/config.py.

    python run_single_ev.py
"""
from ev_charging.config import ScenarioConfig
from ev_charging.ev import EV
from ev_charging.simulation import Simulation


def main() -> None:
    cfg = ScenarioConfig()
    sim = Simulation(cfg)
    sim.add_ev(EV("ev-1", battery_kwh=cfg.battery_kwh, soc=0.0, required_soc=1.0))
    r = sim.run().ev_results[0]

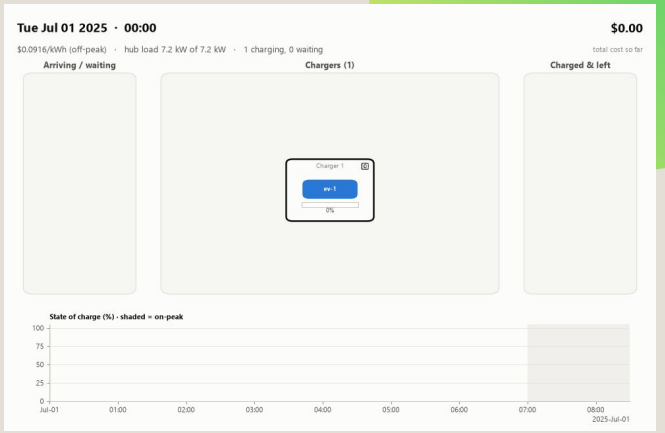
    print(f"Scenario: {cfg.customer_class}, start {cfg.start}, "
          f"{cfg.charger_kw} kW charger, {cfg.battery_kwh} kWh battery")
    print(f"Peak window: {cfg.peak.start_hour:02}-{cfg.peak.end_hour:02}:00"
          f"{' weekdays' if cfg.peak.weekdays_only else ' every day'}")
    print(f"EV {r.ev_id}: SOC {r.start_soc:.0%} -> {r.final_soc:.0%}")
    print(f"  Charged {r.charge_started} -> {r.charge_finished}")
    print(f"  Energy: {r.energy_kwh:.2f} kWh "
          f"(off-peak {r.off_peak_kwh:.2f}, on-peak {r.on_peak_kwh:.2f})")
    print(f"  Cost: ${r.cost:.2f}  (avg ${r.avg_price_per_kwh:.4f}/kWh)")

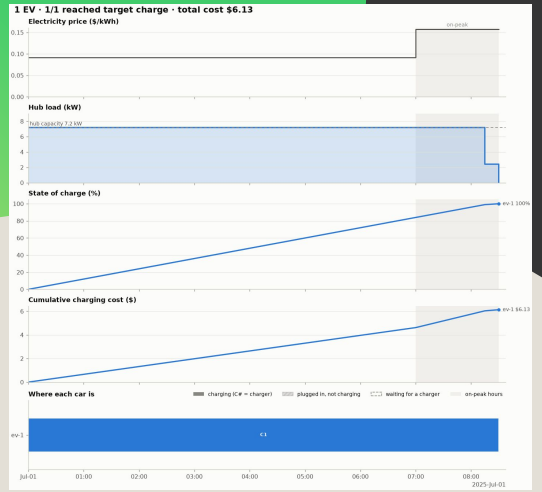

if __name__ == "__main__":
    main()
```

Output:

```
(base) kathrynforrest@Kathryns-MacBook-Air EC601-EV-Charging-Project % python run_single_ev.py
Scenario: residential, start 2025-07-01 00:00:00, 7.2 kW charger, 60.0 kWh battery
Peak window: 7:00-23:00 weekdays
EV ev-1: SOC 0% -> 100%
  Charged 2025-07-01 00:00:00 -> 2025-07-01 08:30:00
  Energy: 60.00 kWh (off-peak 50.40, on-peak 9.60)
  Cost: $6.13  (avg $0.1022/kWh)
(base) kathrynforrest@Kathryns-MacBook-Air EC601-EV-Charging-Project %
```

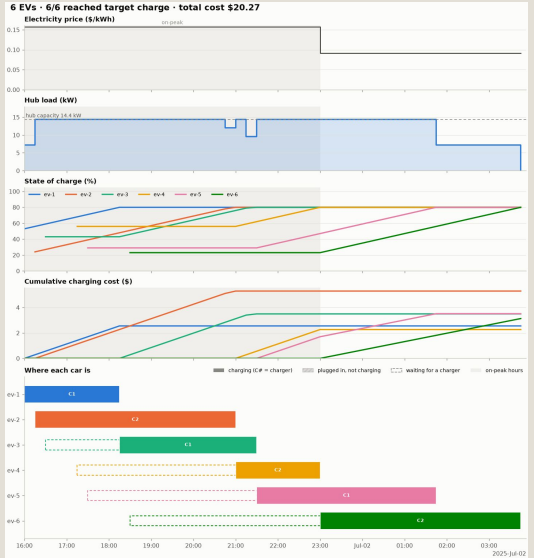
Simulation and Dashboard for 1 EV Charging:





Simulation and Dashboard for an EV Fleet Charging:





Learnings

Sprint 1 Learning Summary

- Use of data, and how it affects simulation results
- Simulation presentation – how to present data and what data is important
- Use of modeling for project planning
- Modules and unit testing while developing framework

Modules and Unit Tests

Modules and unit testing helped to keep code organized and target bugs early.

Project Planning

By creating a model before starting any implementation, we are able to make informed decisions about planning

Data Representation

Deciding what is important to our users and how to display it

Data Collection and Implementation

Importance of dataset on our simulations. Results will not be accurate if the dataset is not.

Sprint 2 Goals

Sprint 1

Developed baseline model and visualization for modules boxed in blue

Sprint 2

Using research papers as basis, implement strategy/strategies for modules boxed in green

